Semiautomated Generation of Logic Rules for Tabular Information in Building Codes to Support Automated Code Compliance Checking

Xiaorui Xue, S.M.ASCE[1]; Jin Wu, S.M.ASCE[2]; and Jiansong Zhang, Ph.D., A.M.ASCE[3]

Abstract: To fully automate building code compliance checking, regulatory requirements need to be automatically extracted and transformed from building codes. Existing regulatory requirement processing efforts mainly focus on building code requirements in text. A more efficient approach for processing regulatory requirements in other parts of building codes, such as tables or charts, remains to be addressed. The ability to process building code requirements in all parts and formats is necessary for an automated code compliance checking system to achieve full coverage of checkable building code requirements. To address this gap, the authors propose a semiautomated information extraction and transformation method. The proposed method can extract building code requirements in tables and convert extracted information to logic rules. Automated code compliance checking systems can utilize the logic rules. The proposed method includes two main steps: (1) tabular information extraction, and (2) rule generation. The tabular information extraction semiautomatically detects the layout of tables, extracts building code requirements from tables, and transforms extracted information to databases. The rule generation step automatically generates logic rules that can be directly executed by logic reasoners. The rule generation step also provides options for users to further refine the generated rules. The development of the tabular information extraction algorithm takes an iterative approach. An experiment was conducted to develop a tabular information extraction algorithm from a section of existing code. The primary version of the algorithm correctly processed 91.67% of tables in a second sample section. After iterative refinements, the updated tabular information extraction algorithm correctly processed all tables in this section. The rule generation algorithm correctly generated logic rules that successfully represented the applicable building code requirements for a testing building, a convenience store in Texas based on the tabular information. **DOI: 10.1061/(ASCE)CP.1943-5487.0001000.** © *2021 American Society of Civil Engineers.*

Introduction

Building information modeling (BIM) developed steadily in the past decade (Noor and Yi 2018). Versatile usage of BIM prompted research of BIM in the architecture, engineering, and construction (AEC) industry for various purposes such as (1) modular construction of residential buildings (Alwisy et al. 2012), (2) construction cost estimation (Lee et al. 2014), (3) collaboration of multidisciplinary teams (Fernando et al. 2013), and (4) energy simulation in a building's early design stage (Kim et al. 2015a). Researchers also combined BIM with other technologies, such as (1) laser scanning (Kim et al. 2015b), (2) radio-frequency identification (RFID) (Fang et al. 2016), (3) virtual reality (VR) (Du et al. 2018), (4) computer vision (Asadi et al. 2019; Ham et al. 2016; Han et al. 2015), and (5) natural language processing (Mutis et al. 2019; Xu and Cai 2020, 2019; Xu et al. 2020; Zhang and El-Gohary 2012a, b) to explore its full potential in the AEC industry. Overall, benefits of BIM include (1) facilitating cooperation among stakeholders (Volk et al. 2014), (2) reducing construction cost and time (Bryde et al. 2013), (3) improving construction safety (Zhang et al. 2013), and (4) decreasing carbon emission and energy consumption of buildings (Gan et al. 2019). Although BIM has a wide range of benefits and BIM-related research was extensive, the adoption and implementation of BIM in the industry are still at an early stage (Noor and Yi 2018). Overall, developed countries have a higher BIM adoption rate than developing countries (Bui et al. 2016). However, even developed countries do not fully adopt BIM. In the United Kingdom, for example, only 54.88% of projects used BIM in the design phase, 51.90% of projects used BIM in the preconstruction phase, and 34.67% of projects used BIM in the construction phase (Eadie et al. 2013). Project participants do not use BIM to its full extent, even when they use BIM in a project. They tend to use BIM to assist their traditional workflows and only in the most expensive and risky part of a project (Migilinskas et al. 2013). In summary, in spite of the diverse applications and benefits of BIM, the BIM adoption rate remains low. Therefore, more automation and user friendliness in BIM-based applications is needed to facilitate the adoption of BIM.

One important such application of BIM is as digital representations of buildings in automated code compliance checking systems (Dimyadi and Amor 2013). Automated code compliance checking systems can work as a driver of BIM adoption (Martins and Abrantes 2010). However, BIM can support automated code compliance checking systems by providing digital representations of building designs (Ismail et al. 2017). Building authorities in multiple countries published automated code compliance checking systems based on BIM such as the Construction and Real Estate

[1]Research Assistant, Automation and Intelligent Construction Lab, School of Construction Management Technology, Purdue Univ., West Lafayette, IN 47907. Email: xue39@purdue.edu

[2]Research Assistant, Automation and Intelligent Construction Lab, School of Construction Management Technology, Purdue Univ., West Lafayette, IN 47907. Email: wu1275@purdue.edu

[3]Assistant Professor, School of Construction Management Technology, Founder and Director, Automation and Intelligent Construction Lab, Purdue Univ., West Lafayette, IN 47907 (corresponding author). ORCID: https://orcid.org/0000-0001-5225-5943. Email: zhan3062@purdue.edu

Note. This manuscript was submitted on December 30, 2020; approved on May 14, 2021; published online on November 1, 2021. Discussion period open until April 1, 2022; separate discussions must be submitted for individual papers. This paper is part of the ***Journal of Computing in Civil Engineering***, © ASCE, ISSN 0887-3801.

NETwork (CORENET) in Singapore (Sing and Zhong 2001), the ByggSok system in Norway (Haraldsen et al. 2004), the DesignCheck in Australia (Ding et al. 2006), and the 3D-4D-BIM program by the US General Services Administration (GSA) (Ho and Matta 2009). In academia, researchers also proposed BIM-based automated code compliance checking systems. For example, Nawari (2011) applied the integration of BIM and SMARTcodes by the International Code Council (ICC) to check the conformance of building elements to various structural codes automatically. Balaban et al. (2012) presented a pilot study that combines BIM and artificial intelligence (AI) to check the compliance of a building model against fire codes automatically. Martins and Monteiro (2013) developed a BIM application for the automated compliance checking of building codes in general. Preidel and Borrmann (2015) presented a flow-based BIM automated code compliance checking system. Zhang and El-Gohary (2017) integrated semantic natural language processing (NLP), logic reasoning, and BIM to develop an automated code checking system for checking with the International Building Code. Bloch and Sacks (2018) explored the possibility of integrating machine learning, rule-based inference making, and BIM to accomplish automated code compliance checking. Fayomi et al. (2018) explored the approach of automating prescriptive compliance process for building energy efficiency in a single BIM software platform. Häußler et al. (2020) integrated BIM and visual programming in code compliance checking of railway designs. In the past decade, the automated code compliance checking domain developed at a fast pace. However, a limited range of checkable codes hinders the wide adoption of automated code compliance checking systems. Few construction industry practitioners will use automated code compliance checking systems that require them to check a significant part of building codes manually. After an extensive literature review, the authors found no previous automated code compliance checking research targeted at automatically processing building code requirements stored in tables. Some previous efforts, such as CORENET (Sing and Zhong 2001), DesignCheck (Ding et al. 2006), and Nawari (2011), who combined SMARTcodes by the ICC and BIM, included tables in the range of checkable building code requirement by manually adding tables to the respective systems. However, almost all building codes contain a large amount of building code requirements in tables. Automated code compliance checking systems that do not cover building code requirements in tables cannot have full building code requirement coverage (Salama and El-Gohary 2016), and pure manual addition of tabular information into an automated code compliance checking system counters efficiency and time/cost savings. Research about tabular information detections and extraction in other domains (Liu et al. 2007; Pinto et al. 2003) concurred with the importance of processing information that is stored in tables in documents. In this research, the authors proposed a new semi-automated information extraction and information transformation method for tables in building codes. The proposed method can process tabular information in building codes to increase the scope of checkable building codes of automated code compliance checking systems.

Background

Existing Work with Table Processing

The demand to extract information from documents that are not in plain textual formats, such as tables and images that are hard for machines to process, is urgent (Corrêa and Zander 2017). Most existing methods take a two-step approach to extract tabular information: (1) table detection, and (2) table substructure identification (i.e., cells, rows, columns) (Paliwal et al. 2019). Challenges in tabular information extraction include (1) reliance on the context of tables to interpret tables, (2) document indexing, (3) database curation, and (4) abbreviation of phases (Shmanina et al. 2016). Table detection algorithms can construct table hierarchies in two approaches: top down and bottom up. In the top-down approach, the algorithm first identifies tables in documents and then slices identified tables into components. In the bottom-up approach, the algorithm first identifies components of tables and then assembles the components to tables (Krüpl and Herzog 2006). Different technologies were developed to process table information for various purposes. For example, Vasileiadis et al. (2017) developed a rule-based, bottom-up tabular information extraction system for access by visually impaired people. Buitelaar et al. (2006) published an ontology-based table processing method to extract information from web pages as part of a multimodal dialog system. Shafait and Smith (2010) used optical character recognition (OCR) technology to process tables with different layouts for analyzing tables in heterogeneous documents. Qasim et al. (2019) treated the table detection problem as a graph problem and generated a graph neural network to detect the structure of tables, which was successfully tested on public table-detection data sets (e.g., UW3, UNLV, and ICDAR 2013). Sinha et al. (2019) used OCR to localize tables in piping and instrumentation diagrams (P&IDs) and used regular expressions to enhance the accuracy of text extraction. Although most researchers treated table detection and table structure identification as two separate steps, Paliwal et al. (2019) proposed TableNet, a neural network with an encoder-decoder structure, to detect table and identify table structure in one unified step. Although existing table processing works reached high accuracy on their respective domains, they did not touch on automation of processing tables in building codes, and data from such tables were still manually interpreted and processed.

Existing Work in Automated Building Code Compliance Checking

From a historical perspective, the traditional building code compliance checking process, or building plan review, is a laborious, time-consuming, and error-prone process that demands automation (Alghamdi et al. 2017; Lee et al. 2018; Preidel and Borrmann 2017). The automation of the code compliance checking process can significantly cut its cost, time, and manual efforts. In the manual code compliance checking process, designers need to wait a long time for building authorities to issue a building permit or ask for further modifications to the design documents, and may have to modify design documents multiple cycles. The plan review process may last a few months (City of San Clemente Ordinance No. 1668). On the other hand, automated code compliance checking systems can return compliance checking results in a much shorter time with a limited need for manual input. Thus, automated building code compliance checking is faster and cheaper than the traditional manual code compliance checking approach.

The automated code compliance checking systems emerged in the 1960s when Fenves introduced decision tables to check the design of steel structures (Fenves 1966). Systems for checking different aspects of building design were then developed over the years. For example, Pauwels et al. (2011) implemented a semantic rule checking environment to check the acoustic performance of buildings. Tan et al. (2010) provided a series of decision tables to check the design of building envelopes. Getuli et al. (2017) developed a BIM-based workflow that checks against the compliance of Italian construction safety and health code. Malsane et al. (2015)

suggested an Industry Foundation Class (IFC)–powered, object-oriented approach to check against fire codes in England and Wales. Bus et al. (2019) developed an ontology-based system to achieve automated compliance checking of semantic rules in French fire safety and accessibility codes. However, existing automated code compliance checking systems only check a limited set of code rules and, according to the authors' literature review, never automatically processed building code requirements in tables.

Database for Table

Storing and manipulating data has been an important topic since the earliest development of computers (Ramakrishnan and Gehrke 2003). Integrated Data Store, developed in the 1960s, was the first general-purpose database management system (DBMS), which standardized the basis of the network data model. Information Management System, on the other hand, was an alternative data representation framework and formed the basis of the hierarchical data model. The current dominant DBMSs utilize relational data models, which were proposed in the 1970s and consolidated in the 1980s. Structured Query Language (SQL) is the standard query language for relational databases. DBMS has the following advantages: (1) data independence, (2) efficient data access, (3) data integrity, (4) data security, (5) efficient data administration, (6) concurrent access, (7) crash recovery, and (8) reduced application development time.

Relational database models store data in tables with unique names (Silberschatz et al. 1997). The rows in a table represent a relationship of a set of values. Relationships between values are represented by tuples, which are sequences of values. The relational data model allows operations like querying, inserting, deleting, and updating of tuples. The relational database model, such as SQL, lacks the ability to take certain actions such as taking input, displaying outputs, or communicating with other programs. These actions must be done through a host language such as C, Java, or Python, with embedded SQL queries. In spite of its limitations, SQL supports a number of useful syntactic features (e.g., relation representation, querying) and has clearly established itself as the standard relational database language (Silberschatz et al. 1997).

Logic Programming

Logic programming aims to connect logic to computer programming (Kowalski 2014). Comparing to the well-established systems of logic, computer programming barely covered the relationship between Turing machines (i.e., an abstract model that simulates the computational ability of computers) and relational algebra. Logic programming remedies the deficiency of computer programming by fitting the generality of logic into the context of computing. Through logic programming, the tasks of defining the problem and solving it can be separated (Spivey 1996). Prolog is a classic programming language that implements logic programming. It uses a top-down method and carries out symbolic reasoning with a logical formula (Kowalski 1979; Spivey 1996). One Prolog program consists of one or more logic clauses. One logic clause consists of one or more predicates. All predicates need to be true to make the logic clause true. For example, for a requirement that says building height should be less than 40 ft, one possible Prolog representation in the form of a logic rule is "*compliance_building_height (Building):-building(Building), building_height(Building_height), has(Building, Building_height), Building_height* <40." The logic clause will be evaluated to true if all predicates to the right of ":-" (i.e., in the body of the rule) are true. The first three predicates [i.e., "*building(Building), building_height(Building_height), has (Building*, *Building_height)*"] check the existence of a building and its building height property. The last predicate ("*building_height* <40") checks if the building height is less than 40 ft. The program can then be fed with building design information in the form of logic facts to check if the building exceeds the maximum allowed building height. If *building_X* has a building height of 50 ft, the corresponding building design facts are "*building(building_X), building_height(building_height), has(building_X, building_height)*, *building*_height == 50." Instantiating the preceding logic rule with these logic facts will lead to the evaluation of the logic rule as false, indicating these building design facts violate the building height requirement.

Proposed Method

In this paper, the authors propose a semiautomated table processing method for tables in building codes. The proposed method takes a two-step approach to process tabular information in building codes: (1) tabular information extraction, and (2) information conversion to logic rules. The proposed method takes building codes and digital models of buildings as inputs, and outputs applicable rules for the building models. The developed method needs to be robust over a wide range of tables, i.e., to be able to process tables in an unseen format. The tabular information extraction method needs to extract building code requirements from tables in building codes and store the extracted information in a structured format. The extraction process needs to reach a very high precision to meet the 100% recall goal of noncompliance detection in automated code compliance checking (Salama and El-Gohary 2016). The format to store extracted building code requirements needs to support easy information access and processing to ensure the performance of the automated code compliance checking system. Integrated methods that directly convert building code tables to logic rules and store these rules in an automated code compliance checking system are the most straightforward and intuitive method to process building code requirements from tables. However, the state-of-the-art integrated methods lack robustness in processing tables in different layouts and the manual effort to maintain integrated methods may not be less than the effort in the manual encoding of building codes per se. The diverse layouts of tables may require customized methods for each table. Frequent updates of building codes will therefore require constant method updates. To address that, the authors propose the separation of information extraction from rule generation to increase the robustness and reduce the maintenance need of the method.

Information Extraction

The proposed method takes a semiautomated approach to extract tabular information from building codes. Users need to collect tables from building codes in digital format and provide them together with some structural information of these input tables. The method then processes one table at a time. Structural information of tables helps the information extraction method identify the layout of the table. For tables with different layouts, the underlying relationships between cells are different. For example, some tables use a single cell to store an entry of building code requirement, and some tables use an entire row to store an entry of building code requirement. Layouts of tables implicitly specify how tables store building code requirements. One type of table, for example, uses a cell and its corresponding row header and column header to represent one requirement to buildings. Another type of table uses all cells in a row and their corresponding column headers to represent

one requirement to buildings. The proposed method uses structural information provided by the users to automatically distinguish layouts of tables and uncover underlying relationships and information inferred by layouts.

The authors took an iterative approach to develop the subalgorithms in the tabular information extraction algorithm, i.e., the subalgorithms are continuously improved until they can correctly extract all tabular information from training data. The basic unit of a table is the cell. Cells can be classified into four types: (1) row header, (2) column header, (3) footnote, and (4) content (Fig. 1). The four types of cells form the body of a table. The finished algorithm can recognize the cell type and connect the information in each cell (e.g., texts, numbers). As a result, the authors developed (1) a header detection subalgorithm to recognize the boundaries of cells for each type, (2) a table layout detection subalgorithm to distinguish layouts of tables, (3) two information transformation subalgorithms to connect contents in the cells, and (4) a rule generation subalgorithm.

The header detection subalgorithm uses the structural information of the table to detect information components. The algorithm requires three inputs from the user for locations of row headers, column headers, and footnotes, respectively. Users then provide (1) the number of columns used for row headers $X1$, (2) the number of rows used for column headers $X2$, and (3) the number of columns used for footnotes $X3$. There may be no footnotes (i.e., zero for $X3$) or row headers (i.e., zero for $X1$). The header detection subalgorithm can then automatically identify the locations of different contents and split the table into different information components according to inputs from the user.

After that, the layout detection subalgorithm distinguishes the layouts of the tables based on their structural information. Tables in building codes have diverse layouts. The authors identified two master layouts based on how the information is organized in a table. Tables with row headers are considered to be in Master Layout 1: a single cell is used to store an entry of building code requirement (Fig. 2). Tables without row headers are considered to be in Master Layout 2: a row of cells is used to store an entry of building code requirement (Fig. 3). Master layouts ensure the robustness of this algorithm and simplify the information extraction process. The layout detection subalgorithm can classify all tables in building codes into these two master layouts depending on whether a table has a row header or not. The authors kept the algorithm simple to ensure the robustness of the entire table information processing.

The end product of this step is a database that stores information from the table. The information conversion subalgorithm connects information in different components of a table and inserts connected information into the database. Each master layout has a customized information conversion subalgorithm. The customized information conversion subalgorithm ensures the correct extraction of information inferred by the layout of tables. Tables in the same master layout use the same information conversion subalgorithm. For tables in the same master layout, variations exist, such as having or not having a column for footnotes, and having or not having a different number of rows in the column header. The information transformation subalgorithms are sufficiently robust to process such variations of tables in the same master layout.

The subalgorithm for Master Layout 1, which is for tables that use a single cell to store an entry of building code requirement,

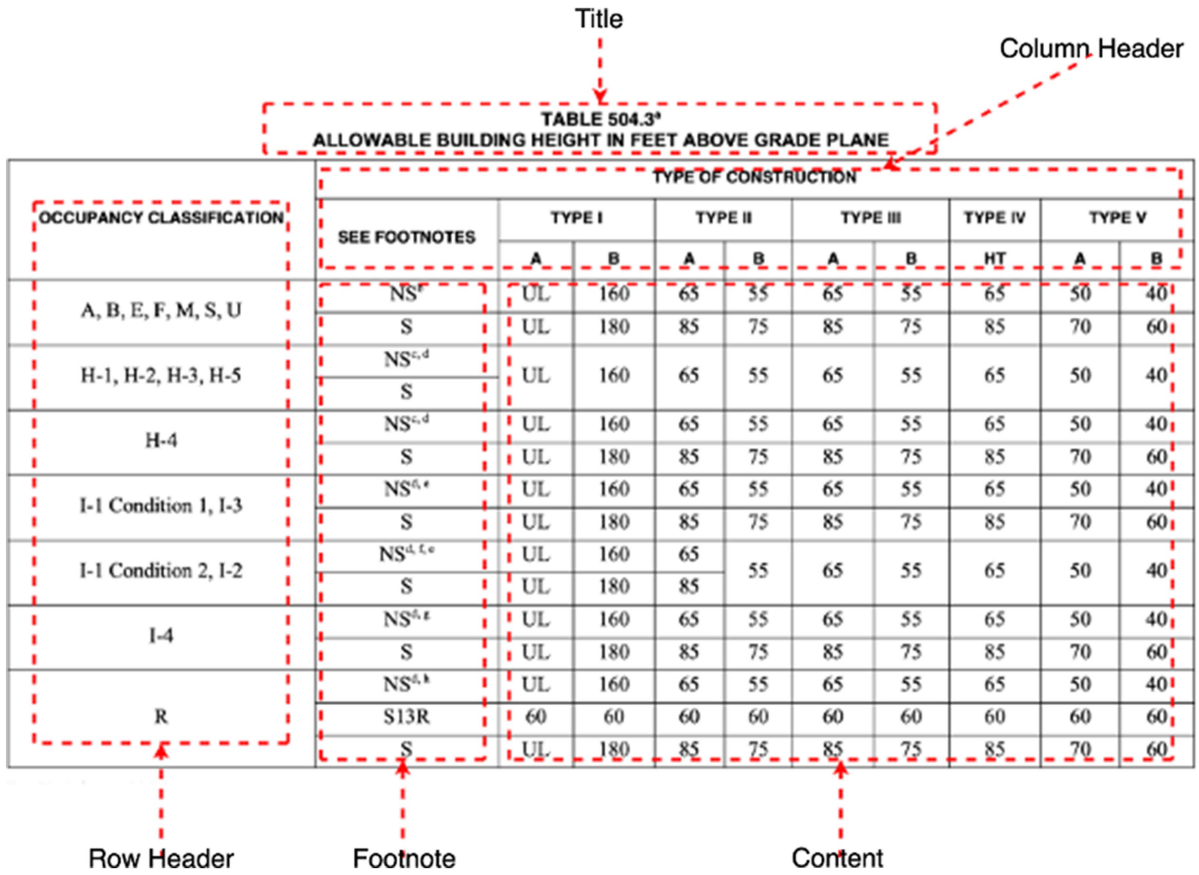

Fig. 1. Example table with the four types of cell components and title. (Reprinted from IBC 2015 with permission from the International Code Council.)

J. Comput. Civ. Eng., 2022, 36(1): 04021033

TABLE 508.4
REQUIRED SEPARATION OF OCCUPANCIES (HOURS)

OCCUPANCY	A, E		I-1[a], I-3, I-4		I-2		R[a]		F-2, S-2[b], U		B[e], F-1, M, S-1		H-1		H-2		H-3, H-4		H-5	
	S	NS	S	NS	S	NS	S	NS	S	NS	S	NS	S	NS	S	NS	S	NS	S	NS
A, E	N	N	1	2	2	NP	1	2	N	1	1	2	NP	NP	3	4	2	3	2	NP
I-1[a], I-3, I-4	—	—	N	N	2	NP	1	NP	1	2	1	2	NP	NP	3	NP	2	NP	2	NP
I-2	—	—	—	—	N	N	2	NP	2	NP	2	NP	NP	NP	3	NP	2	NP	2	NP
R[a]	—	—	—	—	—	—	N	N	1[c]	2[c]	1	2	NP	NP	3	NP	2	NP	2	NP
F-2, S-2[b], U	—	—	—	—	—	—	—	—	N	N	1	2	NP	NP	3	4	2	3	2	NP
B[e], F-1, M, S-1	—	—	—	—	—	—	—	—	—	—	N	N	NP	NP	2	3	1	2	1	NP
H-1	—	—	—	—	—	—	—	—	—	—	—	—	N	NP	NP	NP	NP	NP	NP	NP
H-2	—	—	—	—	—	—	—	—	—	—	—	—	—	—	N	NP	1	NP	1	NP
H-3, H-4	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	1[d]	NP	1	NP
H-5	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	N	NP

Fig. 2. Example table in Master Layout 1. (Reprinted from IBC 2015 with permission from the International Code Council.)

TABLE 509
INCIDENTAL USES

ROOM OR AREA	SEPARATION AND/OR PROTECTION
Furnace room where any piece of equipment is over 400,000 Btu per hour input	1 hour or provide automatic sprinkler system
Rooms with boilers where the largest piece of equipment is over 15 psi and 10 horsepower	1 hour or provide automatic sprinkler system
Refrigerant machinery room	1 hour or provide automatic sprinkler system
Hydrogen fuel gas rooms, not classified as Group H	1 hour in Group B, F, M, S and U occupancies; 2 hours in Group A, E, I and R occupancies.
Incinerator rooms	2 hours and automatic sprinkler system
Paint shops, not classified as Group H, located in occupancies other than Group F	2 hours; or 1 hour and provide automatic sprinkler system
In Group E occupancies, laboratories and vocational shops not classified as Group H	1 hour or provide automatic sprinkler system
In Group I-2 occupancies, laboratories not classified as Group H	1 hour and provide automatic sprinkler system
In ambulatory care facilities, laboratories not classified as Group H	1 hour and provide automatic sprinkler system
Laundry rooms over 100 square feet	1 hour or provide automatic sprinkler system
In Group I-2, laundry rooms over 100 square feet	1 hour
Group I-3 cells and Group I-2 patient rooms equipped with padded surfaces	1 hour
In Group I-2, physical plant maintenance shops	1 hour
In ambulatory care facilities or Group I-2 occupancies, waste and linen collection rooms with containers that have an aggregate volume of 10 cubic feet or greater	1 hour
In other than ambulatory care facilities and Group I-2 occupancies, waste and linen collection rooms over 100 square feet	1 hour or provide automatic sprinkler system
In ambulatory care facilities or Group I-2 occupancies, storage rooms greater than 100 square feet	1 hour
Stationary storage battery systems having a liquid electrolyte capacity of more than 50 gallons for flooded lead-acid, nickel cadmium or VRLA, or more than 1,000 pounds for lithium-ion and lithium metal polymer used for facility standby power, emergency power or uninterruptable power supplies	1 hour in Group B, F, M, S and U occupancies; 2 hours in Group A, E, I and R occupancies.

Fig. 3. Example table in Master Layout 2. (Reprinted from IBC 2015 with permission from the International Code Council.)

connects the cell, its corresponding row header and column header, and its corresponding footnote (if it exists) and generates a command to insert the entry of building code requirement into the database. The subalgorithm for Master Layout 2, which is for tables that use an entire row to store an entry of building code requirement, connects each cell in the row with its corresponding column header and generates a command to insert the entry of building code requirement into the database. Once a command is generated, both

J. Comput. Civ. Eng., 2022, 36(1): 04021033

subalgorithms execute the command to insert building code requirements into the database.

Information Conversion to Logic Rules

The conversion to logic rules follows a semiautomated approach. The conversion process has three parts: raw-rule generation, invariant signatures generation, and information matching.

Raw-Rule Generation

The raw-rule generation utilizes existing semantic NLP-based algorithms that convert building code requirements to Prolog logic rules with placeholders for information from tables (Zhang and El-Gohary 2013, 2015, 2017). The logic rules are generated from building code provisions with reference to tables for depicting a certain required range of values. These existing algorithms lack the ability to process tabular information of building code requirements. Therefore, this step generates logic rules with placeholders for building code requirements in tables. At this step, the information extraction and transformation algorithm by Zhang and El-Gohary (2013, 2015, 2017) was directly adopted, and placeholders were generated in the logic rules for the part dealing with tabular information.

Invariant Signatures Generation

To allow the rules to be compiled with correct quantities and units for the placeholders, the rule generation subalgorithm uses invariant signature (Wu et al. 2021; Wu and Zhang 2019) to decide the configuration of buildings, e.g., occupancy of the building. Each model is processed into invariant signatures, which can represent all the building elements in a uniform way and keep all the needed information for further processing. The invariant signatures include geometrical, locational, and metadata information of the building elements. Table 1 shows an example of the state-of-the-art invariant signature features for structural analysis. In this step, the invariant signatures will be expanded to support the compliance checking use case by adding model-level invariant signatures. Invariant signature allows the proposed algorithm to select the best matching building code requirements from a table for a building and its elements.

Information Matching

The tabular data usually store the regulatory requirements of multiple possible types of buildings. On the other hand, one building usually has a fixed type (e.g., occupation) that corresponds to one entry of the tabular information. The final rule generation subalgorithm can process the digital model of a building and find the corresponding type of the building in the table by querying the databases to get the building code requirements for the building. For example, the occupancy of a building will determine the maximum allowable number of stories of the building [2015 International Building Code (IBC 2015) (ICC 2014)].

With the raw rules with placeholders and the invariant signatures, the rule generation subalgorithm generates final logic rules by an information matching subalgorithm. The information matching subalgorithm will process the invariant signatures to select the corresponding content to fill the placeholders and generate semantically correct B-Prolog (a Prolog system implementation with extensions for programming concurrency, constraints, and interactive graphics) rules (Zhou 2014). For example, the information matching algorithm can process the invariant signature to obtain the occupancy of the building and query the corresponding maximum allowable stories from the database. The query process is based on the invariant signatures. After querying, the rule generation subalgorithm fills the returned values into placeholders of raw logic rules. For example, the maximum allowable stories of the building from the corresponding type of occupancy will be placed in the logic rule. After that, users have the option to manually compile generated rules again to further increase their semantic simplicity if needed.

Experiment

Algorithm Development

The header detection subalgorithm, the layout detection subalgorithm, and two information conversion subalgorithms were developed based on tables (Table 2) in Chapter 5 of IBC 2015 and were tested on tables in Chapter 10 (Table 3) of IBC 2015. Inputs of the developed algorithms were digital tables. Digital tables left less space for errors compared to tables collected as scanned images. The authors manually inspected the extraction results by the algorithm to examine their performance.

After that, the information conversion subalgorithm injected the extracted information into databases. The authors used the SQLite database in the implementation of information conversion subalgorithms (SQLite Consortium 2020). Each table was stored in a

Table 1. Example invariant signatures

Signature name	Signature type	Description	Example values
X-dim	Geometrical	X dimension of the bounding box	57.4
Y-dim	Geometrical	Y dimension of the bounding box	42.0
Origin	Locational	Origin of the placement	(0, 0, 5.0)
X-axis	Locational	Vector of X-axis	(1.0, 0, 0)
Ave-vertices	Metadata	Average number of vertices of each face	4.7

Table 2. Header and cell count of training tables

Table index	Heading	Number of headers	Number of contents
504.3	Allowable building height in feet above grade plane	39	120
504.4	Allowable number of stories above grade plane	102	455
506.2	Allowable area factor (A_i = NS, S1, S13R, or SM, as applicable) in square feet	124	612
508.4	Required separation of occupancies (h)	41	200
509	Incidental uses	2	34

Table 3. Header and cell count of testing tables

Table index	Heading	Number of headers	Number of contents
1004.1.2	Maximum floor area allowances per occupant	2	54
1006.2.1	Spaces with one exit or exit access doorway	20	52
1006.3.1	Minimum number of exits or access to exits per story	2	6
1006.3.2(1)	Stories with one exit or access to one exit for R–2 occupancies	4	8
1006.3.2(2)	Stories with one exit or access to one exit for other occupancies	7	18
1010.1.4.1(1)	Maximum door speed manual revolving doors	2	10
1010.1.4(2)	Maximum door speed automatic or power-operated revolving doors	2	24
1017.2	Exit access travel distance	13	20
1020.1	Corridor fire-resistance rating	11	18
1020.2	Minimum corridor width	2	14
1029.6.2	Capacity for aisles for smoke-protected assembly	11	20
1029.12.2.1	Smoke-protected assembly aisle accessways	16	32

separate database. Two information conversion subalgorithms were developed for the two master layouts. The layout detection subalgorithm selects which information conversion algorithm to use. The information conversion algorithm generates an SQLite insertion command based on the syntax of SQLite and the layout of the table being processed.

Information Conversion to Logic Rule

To test the performance of the rule generation subalgorithm, the authors evaluated it on an IFC model of a real convenience store as the sample building model. The model was using the IFC2x3 standard (Liebich et al. 2008), which was still the most widely used IFC data standard at the time of this study, to ensure the generality of validation. Fig. 4 shows a visualization of the model.

The authors examined the proposed rule generation subalgorithm on the rules in Table 504.3 and Table 504.4 of IBC 2015, which were applicable to the validation case. While Table 506.2 was also applicable, the authors excluded it because it involved the identification and calculation of equations that were out of the scope of this research. The selected rules are shown in Fig. 5.

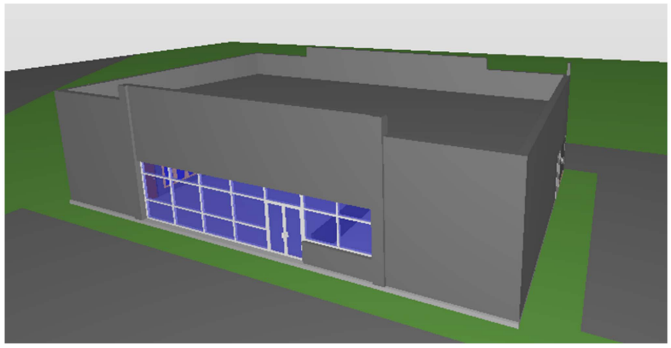

Fig. 4. Visualization of the convenience store model used for testing.

504.3 Height in feet. The maximum height, in feet, of a building shall not exceed the limits specified in Table 504.3.

504.4 Number of stories. The maximum number of stories of a building shall not exceed the limits specified in Table 504.4.

Fig. 5. Selected rules from Chapter 5 of IBC 2015 that have tabular information. (Reprinted from IBC 2015 with permission from the International Code Council.)

To store model-level information, the authors developed invariant signatures to cover the occupancy, construction type, and sprinklered categories. The occupancy category contains all possible occupancy types of a building, such as Group H and Group I. The construction type category contains all possible construction types of a building, such as Type I and Type II. The sprinklered category depicts whether the building is sprinkled, using a Boolean variable.

The authors then processed the model of the convenience store into 58 invariant signatures, which contain 1,363 pieces of information that can be converted into logic facts. Among the 58 invariant signatures, 55 covered element-level information about building components such as wall, slab, roof, window, and door. The remaining three invariant signatures covered model-level information for occupancy, construction type, and sprinkler information.

In the final step, the authors developed an information matching subalgorithm to allow the rules to be filled with the correct content in their placeholders, based on the invariant signatures of the convenience store. The information matching subalgorithm checked each place holder using the invariant signatures and selected the correct header and footer content to modify the raw logic rule into final logic rules with the proper information that is needed for checking the compliance of the building.

Results

The testing results are presented in Table 4. The results showed that the proposed method provided the correct results on 11 testing tables and failed in one. Correctly processed tables are the tables that are correctly converted to databases by the proposed method. The results can be verified manually using queries on the database. The failed table was Table 1006.2.1 (Fig. 6). Therefore, the proposed method processed 91.67% of the tables in the testing data set correctly. The reason that the proposed method failed to provide correct results in Table 1006.2.1 was that this table had four levels of column headers. No table in Chapter 5 of IBC 2015 (i.e., training data) had more than two levels of column headers. The authors then updated the developed algorithm to accommodate tables with different levels of column headers. The updated algorithm was then tested on all testing tables again. The updated algorithm provided correct results on all tables.

The following experiment was further conducted to test if the information extraction subalgorithm correctly preserved the information inferred by the layout of tables and correctly extracted building code requirements in the cells. The accuracy of the algorithm was tested by checking if the generated database returns the correct results when queried. Correct results were where the

Table 4. Results of testing

Table index	Heading	Trained algorithms	Updated algorithms
1004.1.2	Maximum floor area allowances per occupant	Success	Success
1006.2.1	Spaces with one exit or exit access doorway	Fail	Success
1006.3.1	Minimum number of exits or access to exits per story	Success	Success
1006.3.2(1)	Stories with one exit or access to one exit for R–2 occupancies	Success	Success
1006.3.2(2)	Stories with one exit or access to one exit for other occupancies	Success	Success
1010.1.4.1(1)	Maximum door speed manual revolving doors	Success	Success
1010.1.4(2)	Maximum door speed automatic or power-operated revolving doors	Success	Success
1017.2	Exit access travel distance	Success	Success
1020.1	Corridor fire-resistance rating	Success	Success
1020.2	Minimum corridor width	Success	Success
1029.6.2	Capacity for aisles for smoke-protected assembly	Success	Success
1029.12.2.1	Smoke-protected assembly aisle accessways	Success	Success

TABLE 1006.2.1
SPACES WITH ONE EXIT OR EXIT ACCESS DOORWAY

OCCUPANCY	MAXIMUM OCCUPANT LOAD OF SPACE	MAXIMUM COMMON PATH OF EGRESS TRAVEL DISTANCE (feet)		
		Without Sprinkler System (feet)		With Sprinkler System (feet)
		Occupant Load		
		OL ≤ 30	OL > 30	
A[c], E, M	49	75	75	75[a]
B	49	100	75	100[a]
F	49	75	75	100[a]
H-1, H-2, H-3	3	NP	NP	25[b]
H-4, H-5	10	NP	NP	75[b]
I-1, I-2[d], I-4	10	NP	NP	75[a]
I-3	10	NP	NP	100[a]
R-1	10	NP	NP	75[a]
R-2	10	NP	NP	125[a]
R-3[e]	10	NP	NP	125[a]
R-4[e]	10	75	75	125[a]
S[f]	29	100	75	100[a]
U	49	100	75	75[a]

Fig. 6. Table 1006.2.1 from IBC 2015. (Reprinted with permission from the International Code Council.)

corresponding value of a building code requirement in tables can be successfully returned by the query. For example, when the database for Table 1006.2.1 is queried for the maximum occupant load of space of occupancy Type B, it should return 49. The authors queried every entry in the generated databases for every table in the testing data set and reviewed the returned values of every query. In 100% of cases, the query returned correct results. The generated database preserves all information inferred by the layout of the tables. Another reason for the 100% accuracy is the authors used digital tables, instead of scanned tables, as inputs to the information extraction subalgorithm. Errors in recognizing the content of scanned tables were therefore prevented. For example, the algorithm did not suffer from errors in OCR.

Furthermore, the rule generation subalgorithm was tested to generate logic rules with correct parameters based on the invariant signatures of the building model of the real-world convenience store model. The rule generation subalgorithm was tested to generate logic rules for checking the compliance of maximum building height and number of stories of the convenience store. The correct logic rules should represent the meaning of corresponding building code provisions correctly. The raw logic rules with placeholders were checked manually to ensure that the meanings of corresponding building code provisions were correct. Correct final logic rules should have correct building code requirement values (from tables) filled in by the information matching algorithm. The algorithm correctly generated the required final rules for checking the compliance of the convenience store with building code requirements in Tables 504.3 and 504.4 of IBC 2015. The generated rules checked and concluded that the convenience store does not violate the required values in Tables 504.3 and 504.4 of IBC 2015. The detailed testing of the rule generation algorithm is described as follows.

The first step was to apply the state-of-the-art information extraction and transformation algorithms by Zhang and El-Gohary (2013, 2015, 2017) to generate the raw logic rules with placeholders for building code requirements stored in tables that vary with configurations of buildings. An example result is shown in Fig. 7.

Invariant signatures of the convenience store were generated automatically by the state-of-the-art invariant signature generation algorithm (Wu et al. 2021; Wu and Zhang 2019). There were 11 door elements, 2 slab elements, 1 roof element, 27 wall elements, and 14 window elements in the building, as shown in Table 5. Table 5 also gives a few example values of the invariant signature features. Each element is represented uniquely by an invariant

```
compliance_building_height(Building):-
    maximum_height(Maximum_height),
    building(Building),
    in_feet(Maximum_height),
    of(Maximum_height, Building),
    not exceed_the_limits(Maximum_height, Table_504_3).


compliance_building_stories(Building):-
    number_of_stories(Number_of_stories),
    building(Building),
    of(Number_of_stories, Building),
    not exceed_the_limits(Number_of_stories, Table_504_4).
```

Fig. 7. Generated raw rules from Chapter 5 of IBC 2015 with tabular information. (Data from IBC 2015.)

Table 5. Invariant signatures count and examples of building elements

Signature name	Number of elements	Example X-dim	Example origin
Door	11	0.42	(−33.2, 44.1, 338.0)
Slab	2	68.7	(−68.7, 64.71, 338.03)
Roof	1	68.7	(−103.03, 43.96, 350.17)
Wall	27	17.7	(−49.03, 43.96, 338.0)
Window	14	5.36	(−63.44, 80.2, 339.21)

Table 6. Property values based on the invariant signatures of the building

Property	Value
Occupancy	M
Construction type	V-B
Sprinklered	None

signature, which preserves the needed information that will be used for generating the logic rules. In addition to the element-level invariant signatures, the authors also developed model-level invariant signatures. Table 6 shows the building had Occupancy M, Construction Type V-B, and Sprinklered Value None based on model-level invariant signatures. This information was converted into configurations of the convenience store.

Based on the configurations of the convenience store, the developed algorithm was able to find the correct numbers from the table and generate the correct final logic rules. The resulted final logic rules are shown in Fig. 8.

The authors manually compiled generated rules again to make them concise and straightforward. For example, the predicate "not exceed_the_limits" was refined to "<=" because B-Prolog supports the symbol of mathematical inequalities. The finalized rules were improved to use mathematical inequalities to be more concise and machine-readable while staying reader friendly (Fig. 9). This step is optional. Logic rules in Figs. 8 and 9 are equivalent to logic reasoners.

Contributions to the Body of Knowledge

The authors proposed a new method to extend the range of checkable building code requirements of automated building code compliance checking systems to cover tables in building codes. The contributions to the body of knowledge are fourfold. First,

```
compliance_building_height(Building):-
    maximum_height(Maximum_height),
    building(Building),
    in_feet(Maximum_height),
    of(Maximum_height, Building),
    not exceed_the_limits(Maximum_height, 40).


compliance_building_stories(Building):-
    number_of_stories(Number_of_stories),
    building(Building),
    of(Number_of_stories, Building),
    not exceed_the_limits(Number_of_stories, 1).
```

Fig. 8. Final logic rules from Chapter 5 of IBC 2015 with tabular information. (Data from IBC 2015.)

```
compliance_building_height(Building):-
    maximum_height(Maximum_height),
    building(Building),
    in_feet(Maximum_height),
    of(Maximum_height, Building),
    Maximum_height <= 40.


compliance_building_stories(Building):-
    number_of_stories(Number_of_stories),
    building(Building),
    of(Number_of_stories, Building),
    Number_of_stories <= 1.
```

Fig. 9. Refined logic rules from Chapter 5 of IBC 2015 with tabular information. (Data from IBC 2015.)

the extension of checkable building code requirements to tables proves the feasibility of checking nontextual building code requirements in a semiautomated way. Second, this research could help incorporate more building code requirement details into fully automated code compliance checking systems in a more efficient way, comparing to the state of the art. With an enlarged range of checkable building code requirements, an automated code compliance checking system can provide more value to its users, which could lead to a wider adoption of automated building code compliance checking and synergistically facilitating the adoption of BIM. Third, the authors enhanced the robustness of an automated code compliance checking system. By storing the database and generating logic rules on the go, automated code compliance checking systems will benefit from a smaller rule set that has better maintainability compared to a larger one. Last but not least, the authors calculated that 1,542 logic rules can be generated from tables in the training and test data sets, sourced from 17 tables in two chapters of IBC 2015, which has 35 chapters in total. After interpolation, the authors estimated that the proposed method can help complete about 26,985 new rules with tabular information for IBC 2015. The proposed method can therefore significantly expand the range of checkable building code requirements of automated code compliance checking (ACC) systems.

Interface Discussion

Although not the focus of this paper, the proposed method provides a friendly programming interface to support an easy adoption. The information extraction component only needs users to provide a digital table and some structural information of the digital table. To use the information extraction component, users only need to put the digital table and the script of the proposed method in the same folder. When the script is executed, it will prompt users to input information about the table. The output of the information extraction component is database files that are supported in SQL language. The database allows developers to incorporate the tabular information into automated code compliance checking systems more efficiently compared to pure manual interpretation and processing.

Limitations and Future Work

The following limitations are acknowledged. First, the proposed method requires digital tables as inputs and manual conversion or third-party software to process tables from hard copy or images into digital tables. Future versions of the proposed method should incorporate the processing of scanned tables, e.g., using OCR functions. Second, the proposed method requires manual inputs in layout detection. The proposed method cannot detect layouts of tables without such inputs from users even though such inputs are minimal. The authors propose to develop a fully automated layout detection algorithm for tables in building codes in their future work. Third, the rule generation subalgorithm requires manual effort to refine generated rules. Although logic reasoners have no problem in processing unrefined rules, the authors propose to develop refinement algorithms to also generate more human-processable rules automatically in their future work.

Conclusion

This research incorporated tabular information in building codes into automated code compliance checking systems. The proposed table information extraction method achieved a 91.67% success rate in our experiment on tables from IBC 2015. The updated information extraction algorithms could successfully process all tables in the testing data set and correctly preserved information inferred by the layout of tables. The proposed method still requires minor human input, which the authors will further address in their future work.

Data Availability Statement

Some data that support the findings of this study are available from the corresponding author upon reasonable request, including
1. building codes tables used, and
2. logic rules generated.

Acknowledgments

The authors would like to thank Dr. Mark J. Clayton for providing the testing model. The authors would like to thank the National Science Foundation (NSF). This material is based on work supported by the NSF under Grant No. 1827733. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

Alghamdi, A., M. Sulaiman, A. Alghamdi, M. Alhosan, M. Mastali, and J. Zhang. 2017. "Building accessibility code compliance verification using game simulations in virtual reality." In *Proc., Computing in Civil Engineering 2017*, 262–270. Reston, VA: ASCE.

Alwisy, A., M. Al-Hussein, and S. Al-Jibouri. 2012. "BIM approach for automated drafting and design for modular construction manufacturing." In *Proc., Computing in Civil Engineering 2012*, 221–228. Reston, VA: ASCE.

Asadi, P., M. Gindy, and M. Alvarez. 2019. "A machine learning based approach for automatic rebar detection and quantification of deterioration in concrete bridge deck ground penetrating radar B-scan images." *KSCE J. Civ. Eng.* 23 (6): 2618–2627. https://doi.org/10.1007/s12205-019-2012-z.

Balaban, Ö., E. S. Y. Kilimci, and G. Cagdas. 2012. "Automated code compliance checking model for fire egress codes." In *Proc., 30th Int. Conf. on Education and research in Computer Aided Architectural Design in Europe*, 117–125. Prague, Czech: Education and research in Computer Aided Architectural Design in Europe and Faculty of Architecture.

Bloch, T., and R. Sacks. 2018. "Comparing machine learning and rule-based inferencing for semantic enrichment of BIM models." *Autom. Constr.* 91 (Jul): 256–272. https://doi.org/10.1016/j.autcon.2018.03.018.

Bryde, D., M. Broquetas, and J. M. Volm. 2013. "The project benefits of building information modelling (BIM)." *Int. J. Project Manage.* 31 (7): 971–980. https://doi.org/10.1016/j.ijproman.2012.12.001.

Bui, N., C. Merschbrock, and B. E. Munkvold. 2016. "A review of building information modelling for construction in developing countries." *Procedia Eng.* 164: 487–494. https://doi.org/10.1016/j.proeng.2016.11.649.

Buitelaar, P., P. Cimiano, S. Racioppa, and M. Siegel. 2006. "Ontology-based information extraction with soba." In *Proc., Int. Conf. on Language Resources and Evaluation (LREC)*, 2321–2324. Paris: European Language Resources Association.

Bus, N., A. Roxin, G. Picinbono, and M. Fahad. 2019. "Towards French smart building code: Compliance checking based on semantic rules." In *Proc., LDAC2018 6th Linked Data in Architecture and Construction Workshop*, 6–15. London: CEUR Workshop Proceedings.

Corrêa, A. S., and P.-O. Zander. 2017. "Unleashing tabular content to open data: A survey on PDF table extraction methods and tools." In *Proc., 18th Annual Int. Conf. on Digital Government Research*, 54–63. New York: Association for Computing Machinery.

Dimyadi, J., and R. Amor. 2013. "Automated building code compliance checking—Where is it at?" In *Proc., 19th Int. CIB World Building Congress*, 172–185. Ottawa: International Council for Research and Innovation in Building and Construction.

Ding, L., R. Drogemuller, M. Rosenman, and D. Marchant. 2006. "Automating code checking for building designs—DesignCheck." In *Proc., Cooperative Research Centre (CRC) for Construction Innovation*, 1–16. Brisbane, Australia: Cooperative Research Centre for Construction Innovation.

Du, J., Z. Zou, Y. Shi, and D. Zhao. 2018. "Zero latency: Real-time synchronization of BIM data in virtual reality for collaborative decision-making." *Autom. Constr.* 85 (Jan): 51–64. https://doi.org/10.1016/j.autcon.2017.10.009.

Eadie, R., M. Browne, H. Odeyinka, C. McKeown, and S. McNiff. 2013. "BIM implementation throughout the UK construction project lifecycle: An analysis." *Autom. Constr.* 36 (Dec): 145–151. https://doi.org/10.1016/j.autcon.2013.09.001.

Fang, Y., Y. K. Cho, S. Zhang, and E. Perez. 2016. "Case study of BIM and cloud–enabled real-time RFID indoor localization for construction management applications." *J. Constr. Eng. Manage.* 142 (7): 05016003. https://doi.org/10.1061/(ASCE)CO.1943-7862.0001125.

Fayomi, A., F. Castronovo, and R. Akhavian. 2018. "Automating prescriptive compliance process for building energy efficiency through BIM."

In *Proc., 18th Int. Conf. on Construction Applications of Virtual Reality*, edited by R. Amor, 121–132. Auckland, New Zealand: Univ. of Auckland.

Fenves, S. J. 1966. "Tabular decision logic for structural design." *J. Struct. Div.* 92 (6): 473–490. https://doi.org/10.1061/JSDEAG.0001567.

Fernando, T., K.-C. Wu, and M. Bassanino. 2013. "Designing a novel virtual collaborative environment to support collaboration in design review meetings." *J. Inf. Technol. Constr.* 18 (May): 372–396.

Gan, V. J., I. M. Lo, K. T. Tse, C. Wong, J. C. Cheng, and C. M. Chan. 2019. "BIM-based integrated design approach for low carbon green building optimization and sustainable construction." In *Proc., Computing in Civil Engineering 2019: Visualization, Information Modeling, and Simulation—Selected Papers from the ASCE Int. Conf. on Computing in Civil Engineering 2019*, 417–424. Reston, VA: ASCE.

Getuli, V., S. M. Ventura, P. Capone, and A. L. Ciribini. 2017. "BIM-based code checking for construction health and safety." *Procedia Eng.* 196: 454–461. https://doi.org/10.1016/j.proeng.2017.07.224.

Ham, Y., K. K. Han, J. J. Lin, and M. Golparvar-Fard. 2016. "Visual monitoring of civil infrastructure systems via camera-equipped unmanned aerial vehicles (UAVs): A review of related works." *Visualization Eng.* 4 (1): 1. https://doi.org/10.1186/s40327-015-0029-z.

Han, K. K., D. Cline, and M. Golparvar-Fard. 2015. "Formalized knowledge of construction sequencing for visual monitoring of work-in-progress via incomplete point clouds and low-LoD 4D BIMs." *Adv. Eng. Inf.* 29 (4): 889–901. https://doi.org/10.1016/j.aei.2015.10.006.

Haraldsen, M., T. D. Stray, T. Päivärinta, and M. K. Sein. 2004. "Developing e-government portals: From life-events through genres to requirements." In *Proc., 11th Norwegian Conf. on Information Systems*, 44–70. Bonn, Germany: Tapir Academic Publishers.

Häußler, M., S. Esser, and A. Borrmann. 2020. "Code compliance checking of railway designs by integrating BIM, BPMN and DMN." *Autom. Constr.* 121 (Jan): 103427. https://doi.org/10.1016/j.autcon.2020.103427.

Ho, P., and C. Matta. 2009. "Building better: GSA's national 3D-4D-BIM program." *Des. Manage. Rev.* 20 (1): 39–44. https://doi.org/10.1111/j.1948-7169.2009.tb00223.x.

ICC (International Code Council). 2014. *2015 International Building Code*. 3rd printing, 2015. IBC 2015. Washington, DC: ICC. https://codes.iccsafe.org/content/IBC2015.

Ismail, A. S., K. N. Ali, and N. A. Iahad. 2017. "A review on BIM-based automated code compliance checking system." In *Proc., 2017 Int. Conf. on Research and Innovation in Information Systems (ICRIIS)*, 1–6. New York: IEEE.

Kim, J. B., W. Jeong, M. J. Clayton, J. S. Haberl, and W. Yan. 2015a. "Developing a physical BIM library for building thermal energy simulation." *Autom. Constr.* 50 (Feb): 16–28. https://doi.org/10.1016/j.autcon.2014.10.011.

Kim, M.-K., J. C. Cheng, H. Sohn, and C.-C. Chang. 2015b. "A framework for dimensional and surface quality assessment of precast concrete elements using BIM and 3D laser scanning." *Autom. Constr.* 49 (Jan): 225–238. https://doi.org/10.1016/j.autcon.2014.07.010.

Kowalski, R. 1979. *Logic for problem solving*. Madrid, Spain: Ediciones Díaz de Santos.

Kowalski, R. 2014. *Logic for problem solving, revisited*. Norderstedt, Germany: Books on Demand.

Krüpl, B., and M. Herzog. 2006. "Visually guided bottom-up table detection and segmentation in web documents." In *Proc., 15th Int. Conf. on World Wide Web*, 933–934. New York: Association for Computing Machinery.

Lee, S.-K., K.-R. Kim, and J.-H. Yu. 2014. "BIM and ontology-based approach for building cost estimation." *Autom. Constr.* 41 (May): 96–105. https://doi.org/10.1016/j.autcon.2013.10.020.

Lee, Y.-C., P. Ghannad, N. Shang, C. Eastman, and S. Barrett. 2018. "Graphical scripting approach integrated with speech recognition for BIM-based rule checking." In *Proc., Construction Research Congress 2018*, 262–272. Reston, VA: ASCE.

Liebich, T., Y. Adachi, J. Forester, J. Hyvarinen, K. Karstila, and J. Wix. 2008. "IFC2x edition 3." Accessed September 24, 2021. https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/FINAL/HTML.

Liu, Y., K. Bai, P. Mitra, and C. L. Giles. 2007. "Tableseer: Automatic table metadata extraction and searching in digital libraries." In *Proc., 7th ACM/IEEE-CS Joint Conf. on Digital Libraries*, 91–100. New York: Association for Computing Machinery.

Malsane, S., J. Matthews, S. Lockley, P. E. Love, and D. Greenwood. 2015. "Development of an object model for automated compliance checking." *Autom. Constr.* 49 (Jan): 51–58. https://doi.org/10.1016/j.autcon.2014.10.004.

Martins, J., and V. Abrantes. 2010. "Automated code-checking as a driver of BIM adoption." *Int. J. Housing Sci.* 34 (4): 287–295.

Martins, J. P., and A. Monteiro. 2013. "LicA: A BIM based automated code-checking application for water distribution systems." *Autom. Constr.* 29 (Jan): 12–23. https://doi.org/10.1016/j.autcon.2012.08.008.

Migilinskas, D., V. Popov, V. Juocevicius, and L. Ustinovichius. 2013. "The benefits, obstacles and problems of practical BIM implementation." *Procedia Eng.* 57: 767–774. https://doi.org/10.1016/j.proeng.2013.04.097.

Mutis, I., A. Ramachandran, and M. Martinez. 2019. "The BIMbot: A cognitive assistant in the BIM room." In *Proc., 35th CIB W78 2018 Conf. IT in Design, Construction, and Management*, 155–163. Ottawa: International Council for Research and Innovation in Building and Construction.

Nawari, N. O. 2011. "Automating codes conformance in structural domain." In *Proc., Computing in Civil Engineering 2011*, 569–577. Reston, VA: ASCE.

Noor, B. A., and S. Yi. 2018. "Review of BIM literature in construction industry and transportation: Meta-analysis." *Constr. Innovation* 18 (4): 433–452. https://doi.org/10.1108/CI-05-2017-0040.

Paliwal, S. S., D. Vishwanath, R. Rahul, M. Sharma, and L. Vig. 2019. "TableNet: Deep learning model for end-to-end table detection and tabular data extraction from scanned document images." In *Proc., 2019 Int. Conf. on Document Analysis and Recognition (ICDAR)*, 128–133. New York: IEEE.

Pauwels, P., D. Van Deursen, R. Verstraeten, J. De Roo, R. De Meyer, R. Van de Walle, and J. Van Campenhout. 2011. "A semantic rule checking environment for building performance checking." *Autom. Constr.* 20 (5): 506–518. https://doi.org/10.1016/j.autcon.2010.11.017.

Pinto, D., A. McCallum, X. Wei, and W. B. Croft. 2003. "Table extraction using conditional random fields." In *Proc., 26th Annual Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 242–235. New York: Association for Computing Machinery.

Preidel, C., and A. Borrmann. 2015. "Automated code compliance checking based on a visual language and building information modeling." In Vol. 1 of *Proc., Int. Symp. on Automation and Robotics in Construction*, 266–274. Oulu, Finland: IAARC Publications.

Preidel, C., and A. Borrmann. 2017. "Refinement of the visual code checking language for an automated checking of building information models regarding applicable regulations." In *Proc., Computing in Civil Engineering 2017*, 157–165. Reston, VA: ASCE.

Qasim, S. R., H. Mahmood, and F. Shafait. 2019. "Rethinking table recognition using graph neural networks." In *Proc., 2019 Int. Conf. on Document Analysis and Recognition (ICDAR)*, 142–147. New York: IEEE.

Ramakrishnan, R., and J. Gehrke. 2003. *Database management systems*. Boston: McGraw-Hill.

Salama, D. M., and N. M. El-Gohary. 2016. "Semantic text classification for supporting automated compliance checking in construction." *J. Comput. Civ. Eng.* 30 (1): 04014106. https://doi.org/10.1061/(ASCE)CP.1943-5487.0000301.

Shafait, F., and R. Smith. 2010. "Table detection in heterogeneous documents." In *Proc., 9th IAPR Int. Workshop on Document Analysis Systems*, 65–72. New York: Association for Computing Machinery.

Shmanina, T., I. Zukerman, A. L. Cheam, T. Bochynek, and L. Cavedon. 2016. "A corpus of tables in full-text biomedical research publications." In *Proc., 5th Workshop on Building and Evaluating Resources for Biomedical Text Mining (BioTxtM2016)*, 70–79. Osaka, Japan: COLING 2016 Organizing Committee.

Silberschatz, A., H. F. Korth, and S. Sudarshan. 1997. *Database system concepts*. New York: McGraw-Hill.

Sing, T. F., and Q. Zhong. 2001. "Construction and real estate NETwork (CORENET)." *Facilities* 19 (11–12): 419–428. https://doi.org/10.1108/EUM0000000005831.

Sinha, A., J. Bayer, and S. S. Bukhari. 2019. "Table localization and field value extraction in piping and instrumentation diagram images." In *Proc., 2019 Int. Conf. on Document Analysis and Recognition Workshops (ICDARW)*, 26–31. New York: IEEE.

Spivey, J. M. 1996. *An introduction to logic programming through Prolog*. London: Prentice Hall.

SQLite Consortium. 2020. "SQLite home page." Accessed September 24, 2021. https://www.sqlite.org/index.html.

Tan, X., A. Hammad, and P. Fazio. 2010. "Automated code compliance checking for building envelope design." *J. Comput. Civ. Eng.* 24 (2): 203–211. https://doi.org/10.1061/(ASCE)0887-3801(2010)24:2(203).

Vasileiadis, M., N. Kaklanis, K. Votis, and D. Tzovaras. 2017. "Extraction of tabular data from document images." In *Proc., 14th Web for All Conf. on The Future of Accessible Work*, 1–2. New York: Association for Computing Machinery.

Volk, R., J. Stengel, and F. Schultmann. 2014. "Building information modeling (BIM) for existing buildings—Literature review and future needs." *Autom. Constr.* 38 (Mar): 109–127. https://doi.org/10.1016/j.autcon.2013.10.023.

Wu, J., H. L. Sadraddin, R. Ren, J. Zhang, and X. Shao. 2021. "Invariant signatures of architecture, engineering, and construction objects to support BIM interoperability between architectural design and structural analysis." *J. Constr. Eng. Manage.* 147 (1): 04020148. https://doi.org/10.1061/(ASCE)CO.1943-7862.0001943.

Wu, J., and J. Zhang. 2019. "Introducing geometric signatures of architecture, engineering, and construction objects and a new BIM dataset." In *Proc., 2019 ASCE Int. Conf. on Computing in Civil Engineering*, 264–271. Reston, VA: ASCE.

Xu, X., and H. Cai. 2019. "Semantic frame-based information extraction from utility regulatory documents to support compliance checking." In *Advances in informatics and computing in civil and construction engineering*, 223–230. New York: Springer.

Xu, X., and H. Cai. 2020. "Semantic approach to compliance checking of underground utilities." *Autom. Constr.* 109 (Jan): 103006. https://doi.org/10.1016/j.autcon.2019.103006.

Xu, X., K. Chen, and H. Cai. 2020. "Automating utility permitting within highway right-of-way via a generic UML/OCL model and natural language processing." *J. Constr. Eng. Manage.* 146 (12): 04020135. https://doi.org/10.1061/(ASCE)CO.1943-7862.0001936.

Zhang, J., and N. El-Gohary. 2012a. "Automated regulatory information extraction from building codes: Leveraging syntactic and semantic information." In *Proc., Construction Research Congress 2012: Construction Challenges in a Flat World*, 622–632. Reston, VA: ASCE.

Zhang, J., and N. El-Gohary. 2012b. "Extraction of construction regulatory requirements from textual documents using natural language processing techniques." In *Proc., 2012 ASCE Int. Conf. on Computational Civil Engineering*, 453–460. Reston, VA: ASCE.

Zhang, J., and N. El-Gohary. 2013. "Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking." *J. Comput. Civ. Eng.* 30 (2): 04015014. https://doi.org/10.1061/(ASCE)CP.1943-5487.0000346.

Zhang, J., and N. El-Gohary. 2015. "Automated information transformation for automated regulatory compliance checking in construction." *J. Comput. Civ. Eng.* 29 (4): B4015001. https://doi.org/10.1061/(ASCE)CP.1943-5487.0000427.

Zhang, J., and N. M. El-Gohary. 2017. "Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking." *Autom. Constr.* 73 (12): 45–57. https://doi.org/10.1016/j.autcon.2016.08.027.

Zhang, S., J. Teizer, J.-K. Lee, C. M. Eastman, and M. Venugopal. 2013. "Building information modeling (BIM) and safety: Automatic safety checking of construction models and schedules." *Autom. Constr.* 29 (4): 183–195. https://doi.org/10.1016/j.autcon.2012.05.006.

Zhou, N. 2014. "B-Prolog user's manual (version 8.1): Prolog, agent, and constraint programming." Accessed April 1, 2021. http://www.picat-lang.org/bprolog/download/manual.pdf.